Python Activity-Based Learning

1.

Employee Payroll and Deduction Analysis: Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and generate a formatted salary statement.

Sol:

Problem statement

Calculate an employee's gross salary from basic pay, allowances, overtime hours, and incentives, apply progressive deductions, and display the net salary statement.

Objectives

- Calculate HRA, DA, and overtime pay from basic salary.
- Apply progressive tax slabs and PF deduction.
- Display gross salary, deductions, and net salary.

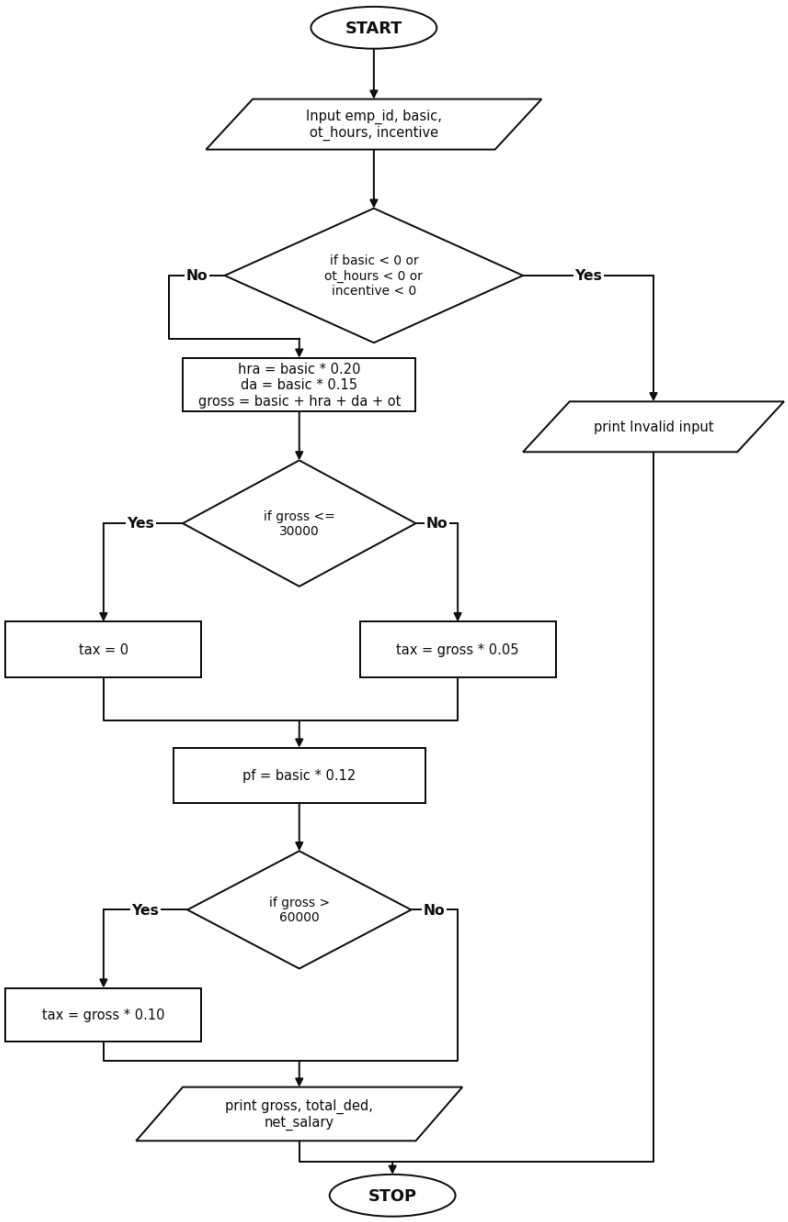
Assumptions

- HRA is 20% of basic pay and DA is 15% of basic pay.
- Overtime rate is Rs. 250 per hour and PF is 12% of basic pay.
- Tax is 0% up to 30,000, 5% up to 60,000, and 10% above 60,000.

Algorithm

1. Start.
2. Read employee ID, name, basic pay, overtime hours, and incentive.
3. Validate that salary and overtime inputs are non-negative.
4. Calculate HRA, DA, overtime pay, and gross salary.
5. Calculate PF and slab-wise tax deduction.
6. Compute net salary and display the salary statement.
7. Stop.

Flowchart



Pseudocode

START

INPUT emp_id, emp_name, basic, ot_hours, incentive

IF basic < 0 OR ot_hours < 0 OR incentive < 0

  PRINT "Invalid input values"

ELSE

  hra = basic * 0.20

  da = basic * 0.15

  ot_pay = ot_hours * 250

```
    gross = basic + hra + da + ot_pay + incentive
    pf = basic * 0.12
    IF gross <= 30000
        tax = 0
    ELSE IF gross <= 60000
        tax = gross * 0.05
    ELSE
        tax = gross * 0.10
    total_ded = pf + tax
    net_salary = gross - total_ded
    PRINT gross, total_ded, net_salary
STOP
```

Source Code

```
emp_id = input("Enter Employee ID: ")
emp_name = input("Enter Employee Name: ")
basic = float(input("Enter Basic Pay: "))
ot_hours = int(input("Enter Overtime Hours: "))
incentive = float(input("Enter Incentive Amount: "))

if basic < 0 or ot_hours < 0 or incentive < 0:
    print("Invalid input values")
else:
    hra = basic * 0.20
    da = basic * 0.15
    ot_pay = ot_hours * 250

    gross = basic + hra + da + ot_pay + incentive
    pf = basic * 0.12

    if gross <= 30000:
        tax = 0
    elif gross <= 60000:
        tax = gross * 0.05
    else:
        tax = gross * 0.10

    total_ded = pf + tax
```

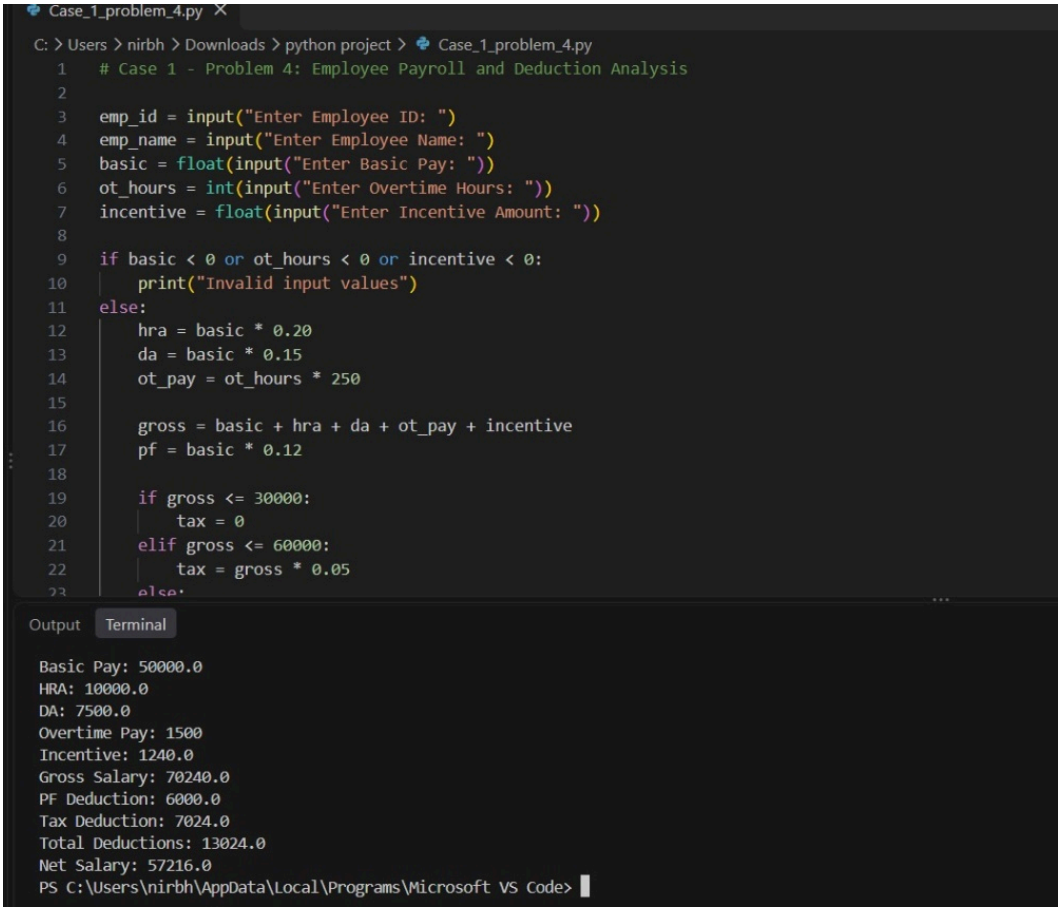
```
net_salary = gross - total_ded

print("--- Salary Statement ---")
print("Employee ID:", emp_id)
print("Employee Name:", emp_name)
print("Basic Pay:", basic)
print("HRA:", hra)
print("DA:", da)
print("Overtime Pay:", ot_pay)
print("Incentive:", incentive)
print("Gross Salary:", gross)
print("PF Deduction:", pf)
print("Tax Deduction:", tax)
print("Total Deductions:", total_ded)
print("Net Salary:", net_salary)
```

Test Cases

1. Enter Employee ID: EMP101
   Enter Employee Name: Ravi Kumar
   Enter Basic Pay: 32000
   Enter Overtime Hours: 8
   Enter Incentive Amount: 2000
   Output:
   Gross Salary: 47200.0
   Total Deductions: 6200.0
   Net Salary: 41000.0
2. Enter Employee ID: EMP102
   Enter Employee Name: Kiran
   Enter Basic Pay: 20000
   Enter Overtime Hours: 4
   Enter Incentive Amount: 1000
   Output:
   Gross Salary: 29000.0
   Total Deductions: 2400.0
   Net Salary: 26600.0

Output



Conclusion

The program calculates employee gross salary, applies progressive tax and PF deductions, and prints the net salary.

2.

Attendance Recovery Computation: Calculate current attendance percentages and determine the minimum consecutive classes required to reach an eligibility threshold under different future attendance scenarios.

Sol:

Problem statement

Calculate a student's current attendance percentage and find the minimum number of consecutive classes needed to reach the 75% eligibility threshold.

Objectives

- Calculate current attendance percentage.
- Use a while loop to find minimum consecutive classes needed for 75%.
- Display future attendance scenario percentages.

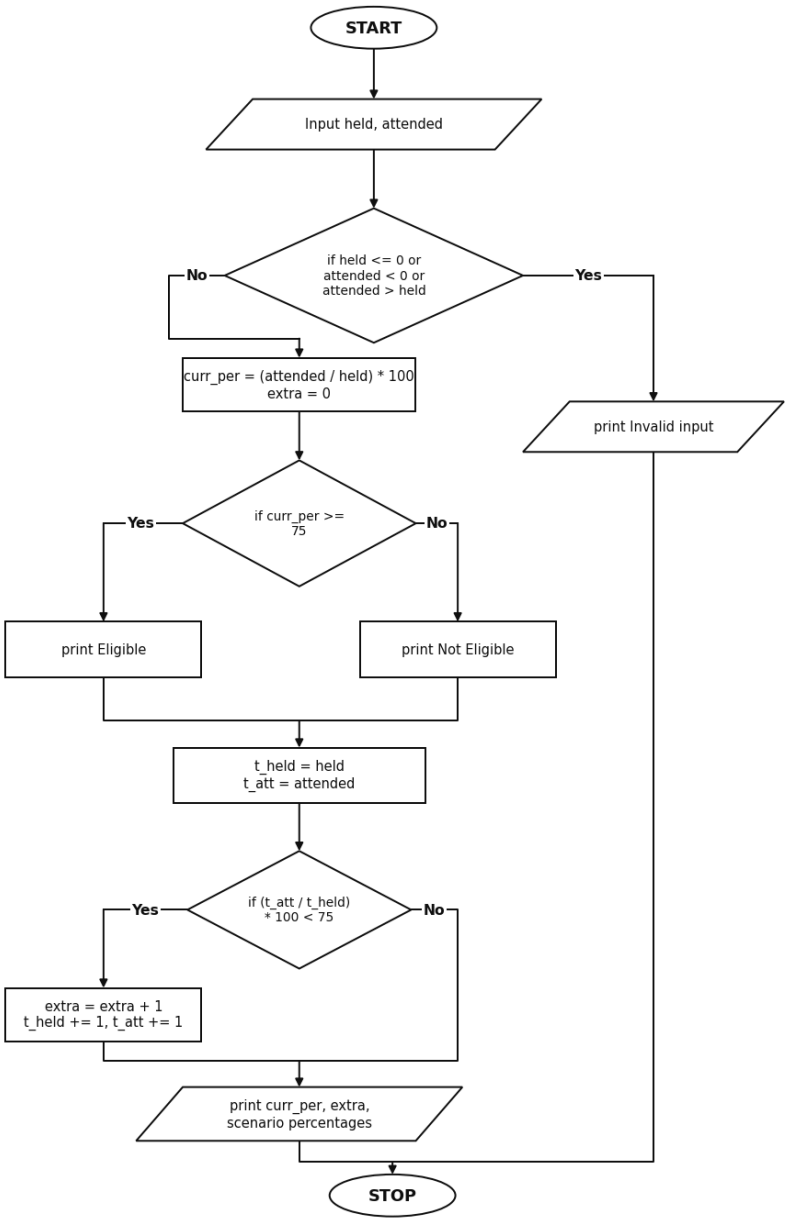
Assumptions

- Total classes held must be greater than zero.
- Classes attended cannot exceed classes held.
- Minimum required attendance for eligibility is 75%.

Algorithm

1. Start.
2. Read total classes held and classes attended.
3. Validate the inputs.
4. Calculate current attendance percentage.
5. While attendance percentage is below 75, add 1 to attended and held classes.
6. Calculate attendance for future scenarios of 5, 10, and 15 classes.
7. Display the results and stop.

Flowchart



Pseudocode

START

INPUT held, attended

IF held <= 0 OR attended < 0 OR attended > held

    PRINT "Invalid attendance inputs"

ELSE

    curr_per = (attended / held) * 100

    PRINT curr_per

    extra = 0

```
    WHILE (attended + extra) / (held + extra) * 100 < 75
        extra = extra + 1
    PRINT extra
    FOR s IN [5, 10, 15]
        new_per = ((attended + s) / (held + s)) * 100
        PRINT s, new_per
STOP
```

Source Code

```
held = int(input("Enter Total Classes Held: "))
attended = int(input("Enter Classes Attended: "))

if held <= 0 or attended < 0 or attended > held:
    print("Invalid attendance inputs")
else:
    curr_per = (attended / held) * 100
    print("Current Attendance Percentage:", round(curr_per, 2))

    if curr_per >= 75:
        print("Status: Eligible for exam")
    else:
        print("Status: Not Eligible (Below 75%)")
        extra = 0
        t_held = held
        t_att = attended

        while (t_att / t_held) * 100 < 75:
            extra = extra + 1
            t_held = t_held + 1
            t_att = t_att + 1

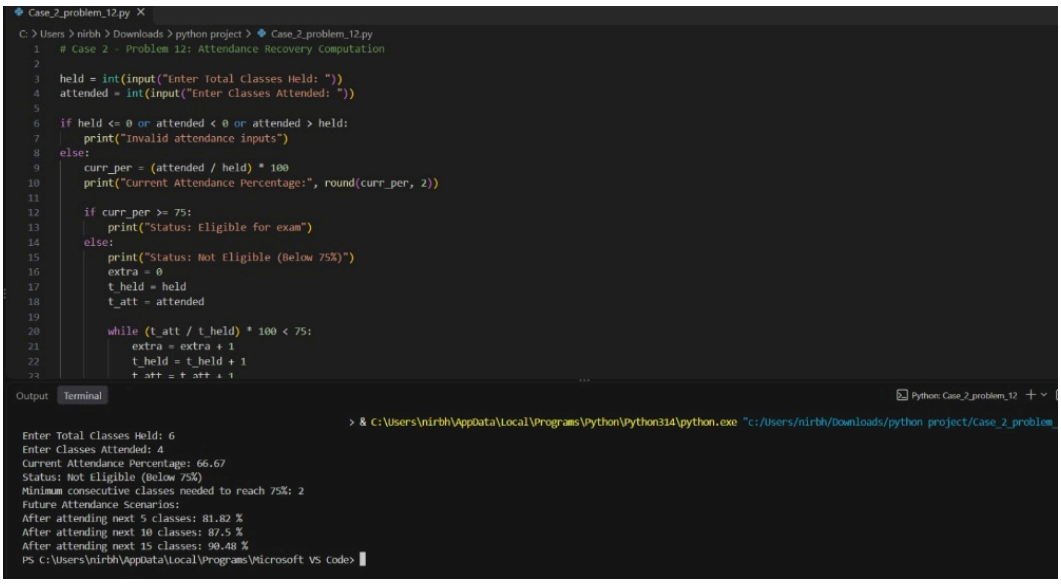
        print("Minimum consecutive classes needed to reach 75%:", extra)

    print("Future Attendance Scenarios:")
    scenarios = [5, 10, 15]
    for s in scenarios:
        new_per = ((attended + s) / (held + s)) * 100
        print("After attending next", s, "classes:", round(new_per, 2), "%")
```

Test Cases

1. Enter Total Classes Held: 40
   Enter Classes Attended: 26
   Output:
   Current Attendance Percentage: 65.0
   Status: Not Eligible (Below 75%)
   Minimum consecutive classes needed to reach 75%: 16
2. Enter Total Classes Held: 50
   Enter Classes Attended: 40
   Output:
   Current Attendance Percentage: 80.0
   Status: Eligible for exam

Output



Conclusion

The program calculates current attendance and uses a loop to find the exact number of consecutive classes needed to reach 75%.

3.

ATM Transaction Sequence Simulation: Process a sequence of deposits and withdrawals, maintain a simulated balance, enforce withdrawal limits, reject invalid transactions, and produce a transaction summary.

Sol:

Problem statement

Process a sequence of ATM deposits and withdrawals, enforce withdrawal and minimum balance limits, reject invalid amounts, and display a transaction summary.

Objectives

- Process deposit and withdrawal transactions sequentially.
- Enforce maximum withdrawal limit and minimum balance rules.
- Count approved and rejected transactions and display closing balance.

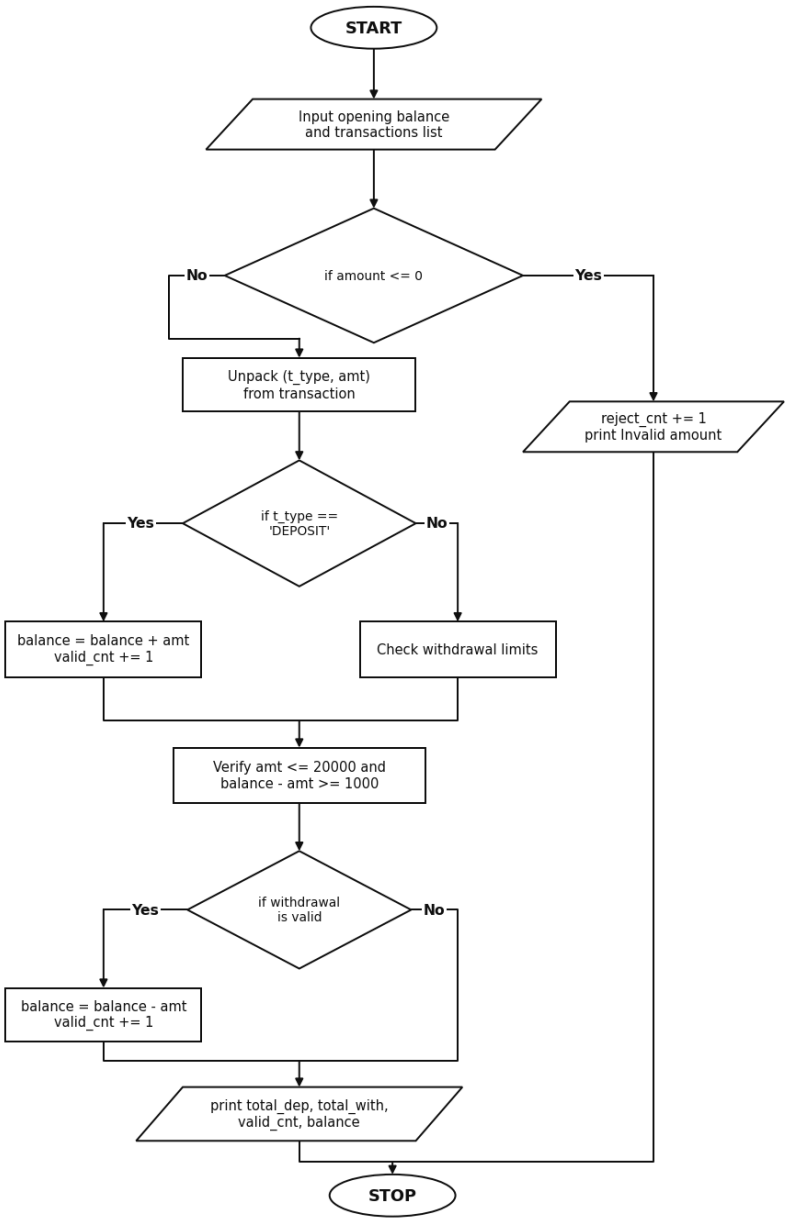
Assumptions

- Maximum per withdrawal transaction is Rs. 20,000.
- Minimum account balance of Rs. 1,000 must be maintained.
- Transaction amounts must be greater than zero.

Algorithm

1. Start.
2. Read the opening account balance.
3. Load the sequence of transaction tuples.
4. For each transaction, unpack transaction type and amount.
5. Reject the transaction if amount <= 0 or withdrawal rules are violated.
6. Otherwise update balance and increment approved count.
7. Display final balance and transaction counts.
8. Stop.

Flowchart



Pseudocode

START

INPUT balance

SET valid_cnt = 0, reject_cnt = 0

FOR each (t_type, amt) IN transactions

  IF amt <= 0

    reject_cnt = reject_cnt + 1

  ELSE IF t_type == "DEPOSIT"

    balance = balance + amt

```
        valid_cnt = valid_cnt + 1
    ELSE IF t_type == "WITHDRAW"
        IF amt > 20000 OR (balance - amt) < 1000
            reject_cnt = reject_cnt + 1
        ELSE
            balance = balance - amt
            valid_cnt = valid_cnt + 1
PRINT total_dep, total_with, valid_cnt, reject_cnt, balance
STOP
```

Source Code

```
balance = float(input("Enter Opening Balance: "))

transactions = [
    ("DEPOSIT", 5000),
    ("WITHDRAW", 3000),
    ("WITHDRAW", 25000),
    ("WITHDRAW", 12000),
    ("DEPOSIT", -500),
    ("DEPOSIT", 2500)
]

total_dep = 0
total_with = 0
valid_cnt = 0
reject_cnt = 0

print("Processing Transactions:")
for txn in transactions:
    t_type, amt = txn

    if amt <= 0:
        print(t_type, amt, "-> Rejected (Invalid amount)")
        reject_cnt = reject_cnt + 1
    elif t_type == "DEPOSIT":
        balance = balance + amt
        total_dep = total_dep + amt
        valid_cnt = valid_cnt + 1
```

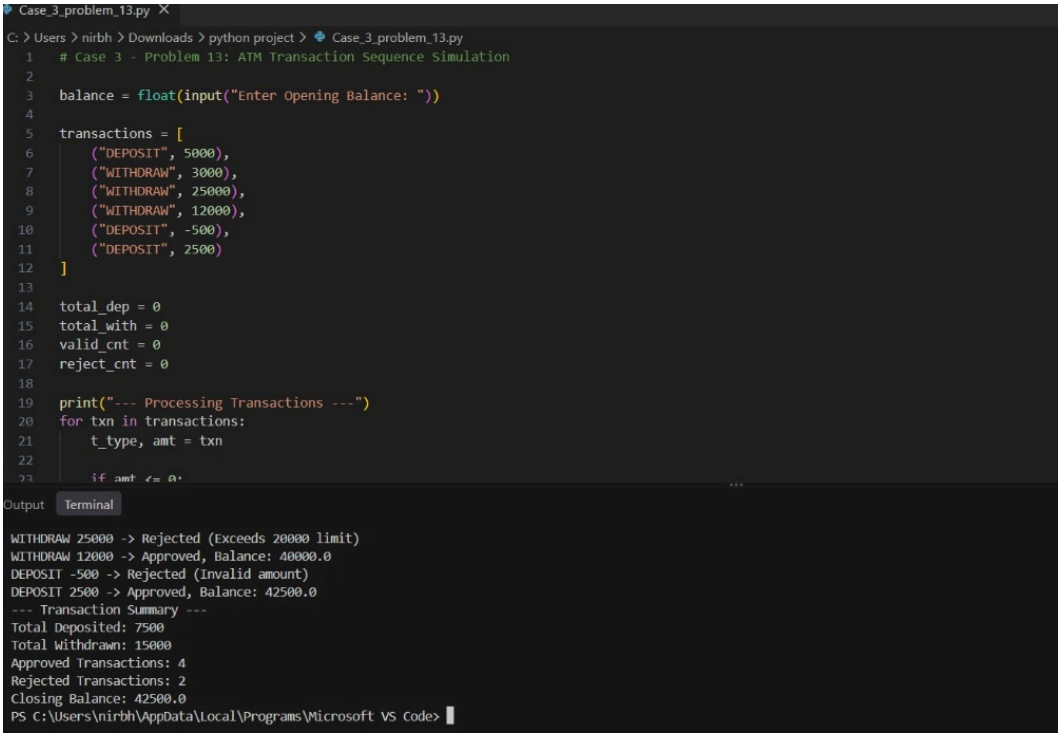
```
        print("DEPOSIT", amt, "-> Approved, Balance:", balance)
    elif t_type == "WITHDRAW":
        if amt > 20000:
            print("WITHDRAW", amt, "-> Rejected (Exceeds 20000 limit)")
            reject_cnt = reject_cnt + 1
        elif (balance - amt) < 1000:
            print("WITHDRAW", amt, "-> Rejected (Minimum balance 1000 required)")
            reject_cnt = reject_cnt + 1
        else:
            balance = balance - amt
            total_with = total_with + amt
            valid_cnt = valid_cnt + 1
            print("WITHDRAW", amt, "-> Approved, Balance:", balance)

print("--- Transaction Summary ---")
print("Total Deposited:", total_dep)
print("Total Withdrawn:", total_with)
print("Approved Transactions:", valid_cnt)
print("Rejected Transactions:", reject_cnt)
print("Closing Balance:", balance)
```

Test Cases

1. Enter Opening Balance: 10000
   Output:
   Total Deposited: 7500
   Total Withdrawn: 3000
   Approved Transactions: 3
   Rejected Transactions: 3
   Closing Balance: 14500.0
2. Enter Opening Balance: 20000
   Output:
   Total Deposited: 7500
   Total Withdrawn: 15000
   Approved Transactions: 4
   Rejected Transactions: 2
   Closing Balance: 12500.0

Output



Conclusion

The program simulates ATM deposits and withdrawals using loops and conditions while enforcing withdrawal and minimum balance limits.

4.

String-Based Examination Answer Evaluation: Compare answer strings against an answer key, calculate correct, incorrect, and unanswered responses, apply negative marking, and produce student-wise results.

Sol:

Problem statement

Compare student answer strings against a 10-question master answer key, apply negative marking, and display student-wise marks and pass/fail results.

Objectives

- Compare answer strings character by character with the answer key.
- Count correct, wrong, and unanswered questions for each student.
- Calculate final marks with negative marking and classify pass or fail.

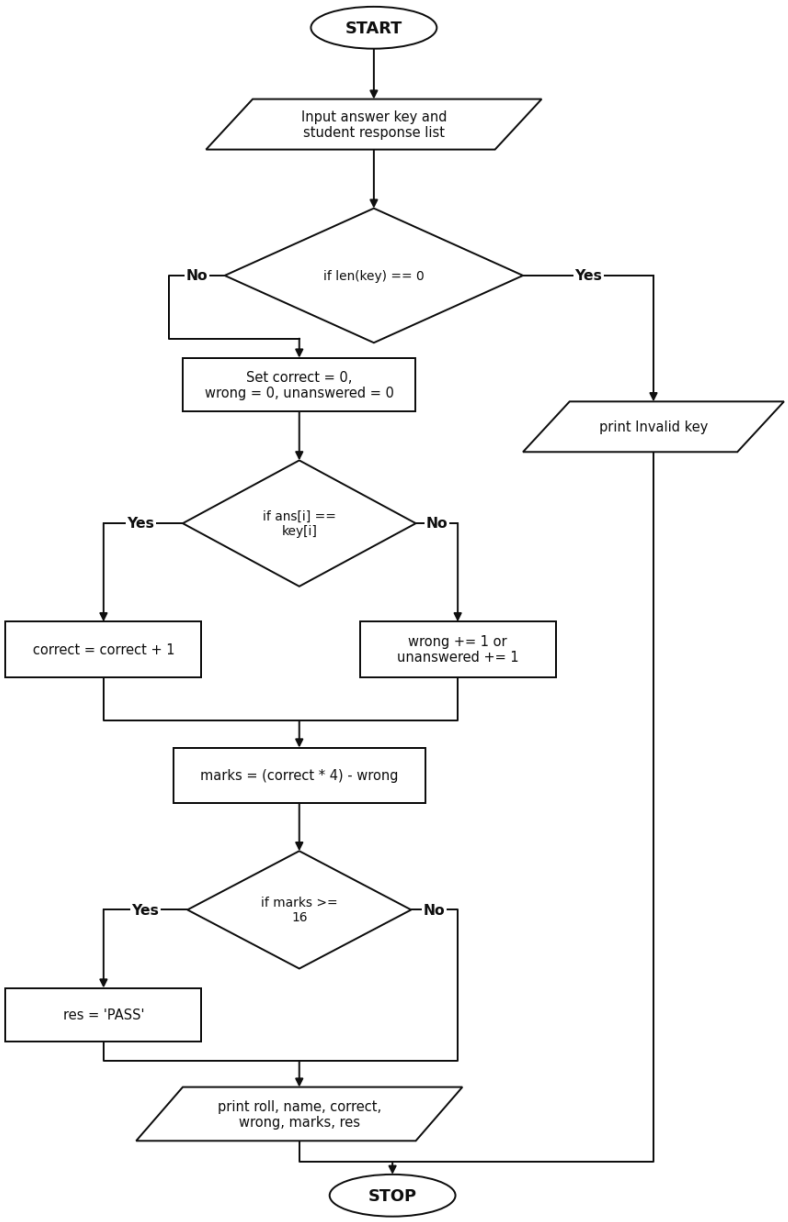
Assumptions

- Each correct response awards +4 marks and each wrong response deducts 1 mark.
- Unanswered questions are marked with 'X' and carry 0 marks.
- Minimum passing score is 16 marks.

Algorithm

1. Start.
2. Read the 10-character master answer key.
3. For each student record, initialize correct, wrong, and unanswered counts to 0.
4. Compare each character of the student response with the answer key.
5. Increment unanswered for 'X', correct for a match, and wrong otherwise.
6. Compute marks = (correct * 4) - wrong and check if marks >= 16.
7. Display each student's score and result.
8. Stop.

Flowchart



Pseudocode

START

INPUT key

FOR each (roll, name, ans) IN students

  SET correct = 0, wrong = 0, unanswered = 0

  FOR i = 0 TO len(key) - 1

    IF ans[i] == "X"

      unanswered = unanswered + 1

    ELSE IF ans[i] == key[i]

```
            correct = correct + 1
        ELSE
            wrong = wrong + 1
    marks = (correct * 4) - wrong
    IF marks >= 16
        res = "PASS"
    ELSE
        res = "FAIL"
    PRINT roll, name, correct, wrong, unanswered, marks, res
STOP
```

Source Code

```
key = input("Enter 10-question Answer Key (e.g. ABCDABCDAB): ").upper()

students = [
    ("26A01", "Ananya", "ABCDABXXAB"),
    ("26A02", "Kiran", "ABCCABDDAB"),
    ("26A03", "Meena", "ADCDXXCDCB"),
    ("26A04", "Rohit", "DCBAXXXXDC")
]

print("--- Exam Result Evaluation ---")
for st in students:
    roll, name, ans = st
    correct = 0
    wrong = 0
    unanswered = 0

    for i in range(len(key)):
        if ans[i] == "X":
            unanswered = unanswered + 1
        elif ans[i] == key[i]:
            correct = correct + 1
        else:
            wrong = wrong + 1

    marks = (correct * 4) - (wrong * 1)
    if marks >= 16:
```

```
        res = "PASS"
    else:
        res = "FAIL"

    print(roll, name, "-> Correct:", correct, "Wrong:", wrong, "Unanswered:", unanswered, "Marks:", marks, "Result:", res)
```

Test Cases

1. Enter 10-question Answer Key: ABCDABCDAB

   Output:

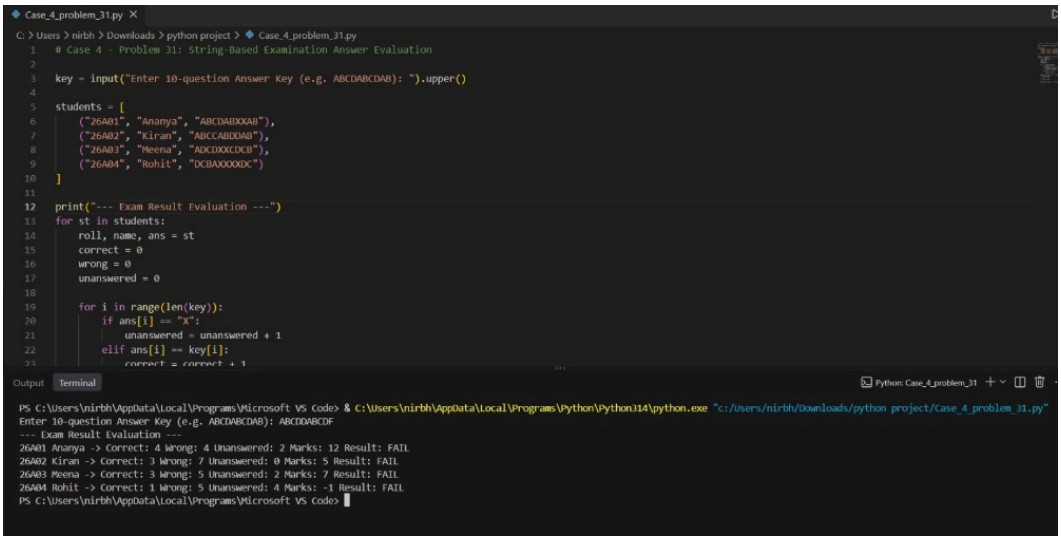
   26A01 Ananya -> Correct: 8 Wrong: 0 Unanswered: 2 Marks: 32 Result: PASS

   26A02 Kiran -> Correct: 8 Wrong: 2 Unanswered: 0 Marks: 30 Result: PASS

   26A03 Meena -> Correct: 6 Wrong: 2 Unanswered: 2 Marks: 22 Result: PASS

   26A04 Rohit -> Correct: 0 Wrong: 6 Unanswered: 4 Marks: -6 Result: FAIL

Output



Conclusion

The program evaluates student answer strings using string indexing and loops, applies negative marking, and displays student-wise results.

5.

Railway Fare and Reservation Calculator: Calculate fares for multiple passengers using distance slabs, travel classes, age-based concessions, reservation fees, and specified cancellation penalties.

Sol:

Problem statement

Calculate railway ticket fares for passengers based on travel distance slabs, class multipliers, age concessions, reservation fees, and cancellation penalties.

Objectives

- Calculate distance-based base fare.
- Apply travel class multipliers and age concessions.
- Compute reservation charges and cancellation penalties.

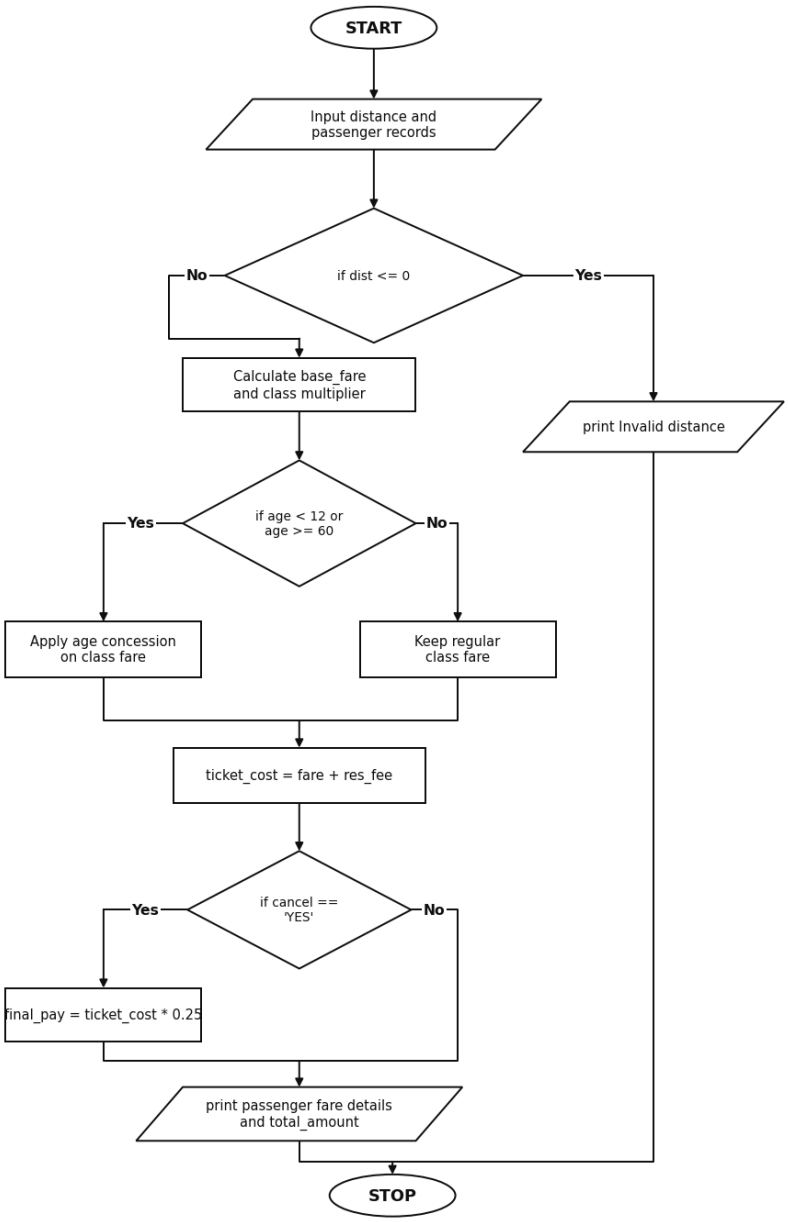
Assumptions

- Base rate is Rs. 1.5/km up to 300 km and Rs. 1.2/km above 300 km.
- Children under 12 get 50% concession and seniors 60+ get 40% concession.
- Cancelled tickets are charged a 25% cancellation penalty.

Algorithm

1. Start.
2. Read journey distance and calculate slab base fare.
3. For each passenger, read name, age, class, and cancellation status.
4. Apply class multiplier and reservation fee.
5. Apply age concession for children or senior citizens.
6. If ticket is cancelled, compute 25% cancellation charge.
7. Display passenger-wise fares and total payable amount.
8. Stop.

Flowchart



Pseudocode

START

INPUT dist

IF dist <= 300

  base_fare = dist * 1.5

ELSE

  base_fare = dist * 1.2

FOR each (name, age, p_class, cancel) IN passengers

  SET mult and res_fee based on p_class

```
    fare = base_fare * mult
    IF age < 12
        fare = fare * 0.50
    ELSE IF age >= 60
        fare = fare * 0.60
    ticket_cost = fare + res_fee
    IF cancel == "YES"
        final_pay = ticket_cost * 0.25
    ELSE
        final_pay = ticket_cost
    PRINT name, p_class, ticket_cost, final_pay
STOP
```

Source Code

```
dist = float(input("Enter Travel Distance (km): "))

if dist <= 300:
    base_fare = dist * 1.5
else:
    base_fare = dist * 1.2

passengers = [
    ("Suresh", 34, "3AC", "NO"),
    ("Lakshmi", 65, "3AC", "NO"),
    ("Chintu", 9, "SL", "NO"),
    ("Vikram", 42, "2AC", "YES")
]

total_amount = 0
print("--- Passenger Fare Details ---")

for p in passengers:
    name, age, p_class, cancel = p

    if p_class == "SL":
        mult = 1.0
        res_fee = 40
    elif p_class == "3AC":
```

```
        mult = 2.0
        res_fee = 60
    else:
        mult = 3.0
        res_fee = 80

    fare = base_fare * mult

    if age < 12:
        fare = fare * 0.50
    elif age >= 60:
        fare = fare * 0.60

    ticket_cost = fare + res_fee

    if cancel == "YES":
        final_pay = ticket_cost * 0.25
        status = "Cancelled"
    else:
        final_pay = ticket_cost
        status = "Confirmed"

    total_amount = total_amount + final_pay
    print(name, "| Class:", p_class, "| Booked:", round(ticket_cost, 2), "| Status:", status, "| Payable:",
round(final_pay, 2))

print("Total Amount Payable:", round(total_amount, 2))
```

Test Cases

1. Enter Travel Distance (km): 450

   Output:

   Suresh | Class: 3AC | Booked: 1140.0 | Status: Confirmed | Payable: 1140.0

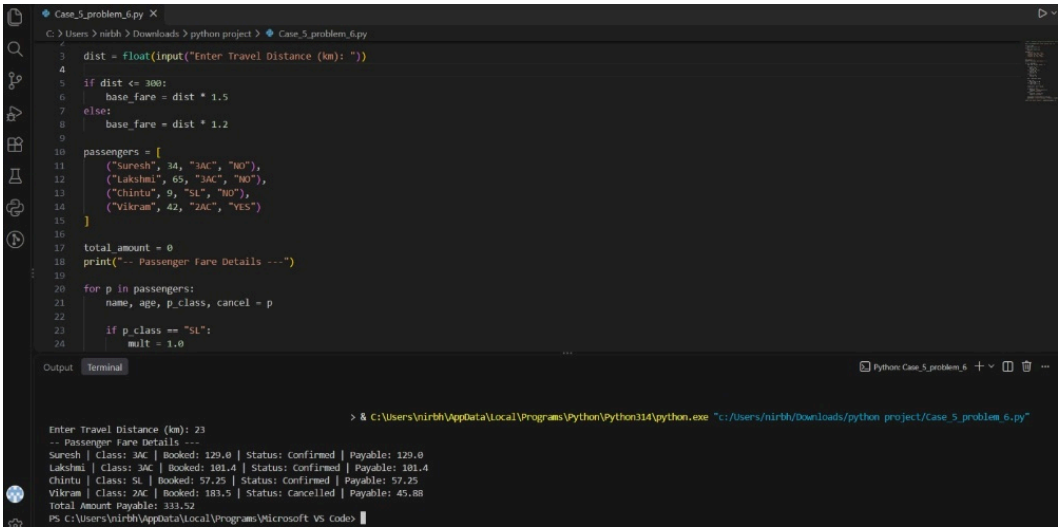
   Lakshmi | Class: 3AC | Booked: 708.0 | Status: Confirmed | Payable: 708.0

   Chintu | Class: SL | Booked: 310.0 | Status: Confirmed | Payable: 310.0

   Vikram | Class: 2AC | Booked: 1700.0 | Status: Cancelled | Payable: 425.0

   Total Amount Payable: 2583.0

Output



Conclusion

The program calculates railway fares across travel classes, applies age concessions and cancellation penalties, and prints the total fare.

6.

Warehouse Inventory Reconciliation: Compare opening stock, incoming quantities, dispatched quantities, and closing stock across multiple products; detect discrepancies and calculate replenishment requirements.

Sol:

Problem statement

Compare expected closing stock with physical closing stock across warehouse products, identify stock discrepancies, and calculate replenishment units needed.

Objectives

- Compute expected closing stock from opening, incoming, and dispatched quantities.
- Detect stock shortages and surpluses.
- Calculate replenishment units required to meet target stock.

Assumptions

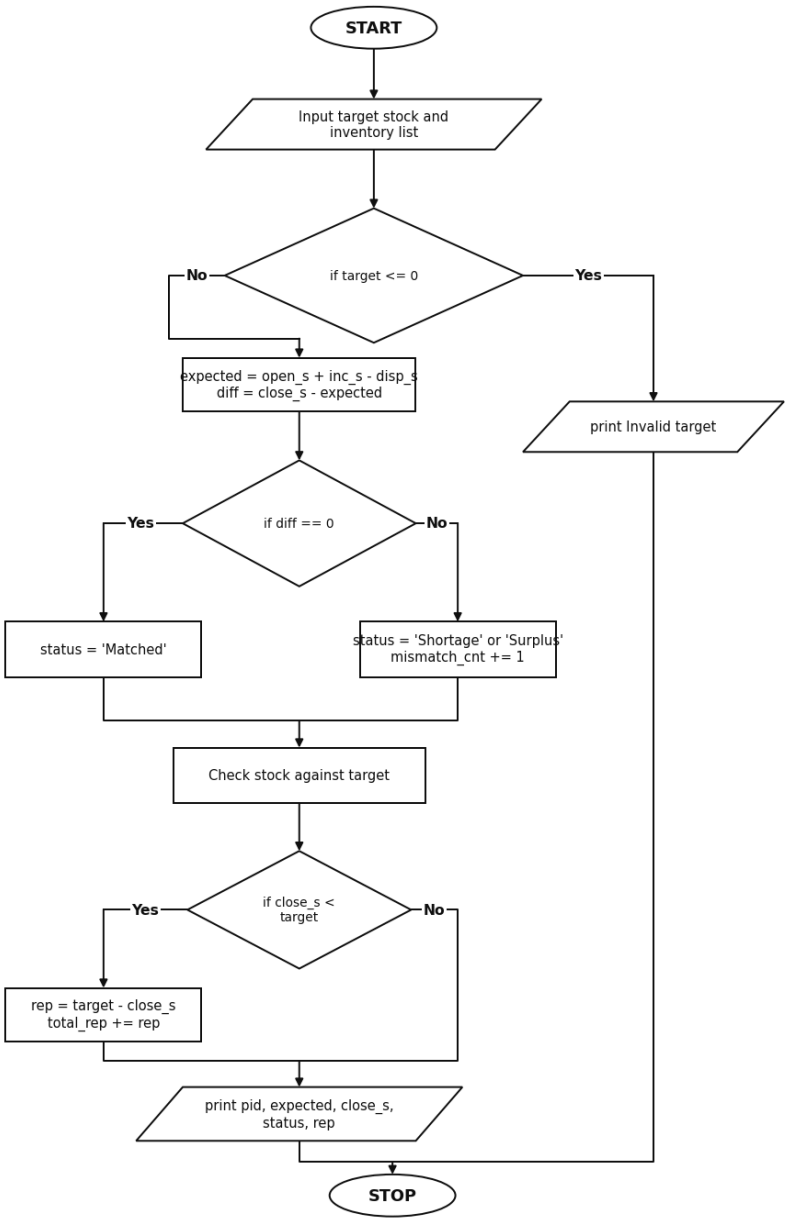
- Expected closing stock equals opening + incoming - dispatched.
- Discrepancy occurs whenever physical closing stock differs from expected closing stock.

- Products below the target stock level require replenishment.

Algorithm

1. Start.
2. Read the minimum target stock level.
3. For each product record in the nested inventory list, compute expected stock.
4. Compare actual closing stock with expected stock to detect shortage or surplus.
5. If actual closing stock is below target, calculate replenishment needed.
6. Display product-wise reconciliation status and total replenishment.
7. Stop.

Flowchart



Pseudocode

START

INPUT target

FOR each [pid, pname, open_s, inc_s, disp_s, close_s] IN inventory

    expected = open_s + inc_s - disp_s

    diff = close_s - expected

    IF diff == 0

        status = "Matched"

    ELSE IF diff < 0

```
        status = "Shortage"
    ELSE
        status = "Surplus"
    IF close_s < target
        rep = target - close_s
    ELSE
        rep = 0
    PRINT pid, pname, expected, close_s, status, rep
STOP
```

Source Code

```
target = int(input("Enter Minimum Stock Target: "))

# [Product ID, Name, Opening, Incoming, Dispatched, Closing]
inventory = [
    ["P101", "Keyboards", 120, 50, 60, 110],
    ["P102", "Monitors", 80, 40, 50, 65],
    ["P103", "Mouse", 150, 30, 40, 140],
    ["P104", "Cables", 90, 20, 35, 75],
    ["P105", "Headsets", 60, 25, 30, 58]
]

mismatch_cnt = 0
total_rep = 0

print("--- Inventory Reconciliation Report ---")
for item in inventory:
    pid = item[0]
    pname = item[1]
    open_s = item[2]
    inc_s = item[3]
    disp_s = item[4]
    close_s = item[5]

    expected = open_s + inc_s - disp_s
    diff = close_s - expected

    if diff == 0:
```

```
        status = "Matched"
    elif diff < 0:
        status = "Shortage"
        mismatch_cnt = mismatch_cnt + 1
    else:
        status = "Surplus"
        mismatch_cnt = mismatch_cnt + 1

    if close_s < target:
        rep = target - close_s
    else:
        rep = 0

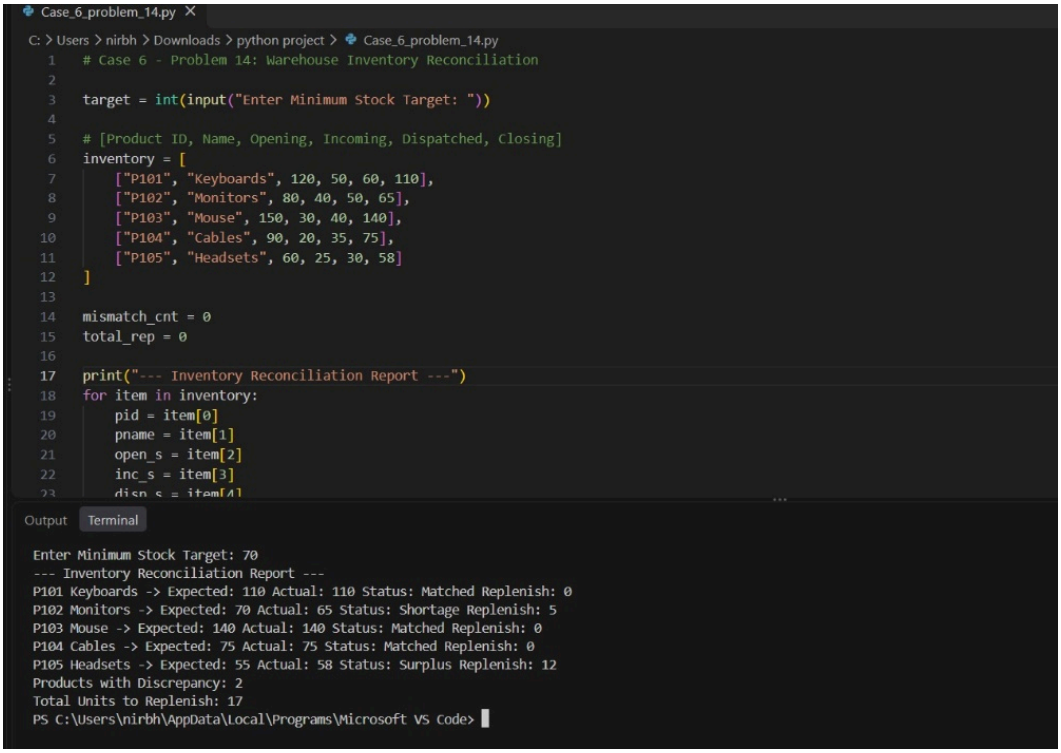
    total_rep = total_rep + rep
    print(pid, pname, "-> Expected:", expected, "Actual:", close_s, "Status:", status, "Replenish:", rep)

print("Products with Discrepancy:", mismatch_cnt)
print("Total Units to Replenish:", total_rep)
```

Test Cases

1. Enter Minimum Stock Target: 100
   Output:
   P101 Keyboards -> Expected: 110 Actual: 110 Status: Matched Replenish: 0
   P102 Monitors -> Expected: 70 Actual: 65 Status: Shortage Replenish: 35
   P103 Mouse -> Expected: 140 Actual: 140 Status: Matched Replenish: 0
   P104 Cables -> Expected: 75 Actual: 75 Status: Matched Replenish: 25
   P105 Headsets -> Expected: 55 Actual: 58 Status: Surplus Replenish: 42
   Products with Discrepancy: 2
   Total Units to Replenish: 102

Output



Conclusion

The program reconciles warehouse stock records using nested lists, flags inventory discrepancies, and calculates replenishment quantities.

7.

Traffic Violation Penalty Calculator: Process multiple violations using vehicle category, offence type, repeat-offence status, and predefined fine rules; calculate the final penalty and identify disqualification conditions.

Sol:

Problem statement

Calculate traffic violation fines based on offence type, vehicle category, and repeat-offence count, and flag licenses for disqualification.

Objectives

- Assign base fines according to traffic offence type.
- Apply heavy-vehicle and repeat-offender multipliers.
- Identify drivers reaching the disqualification threshold.

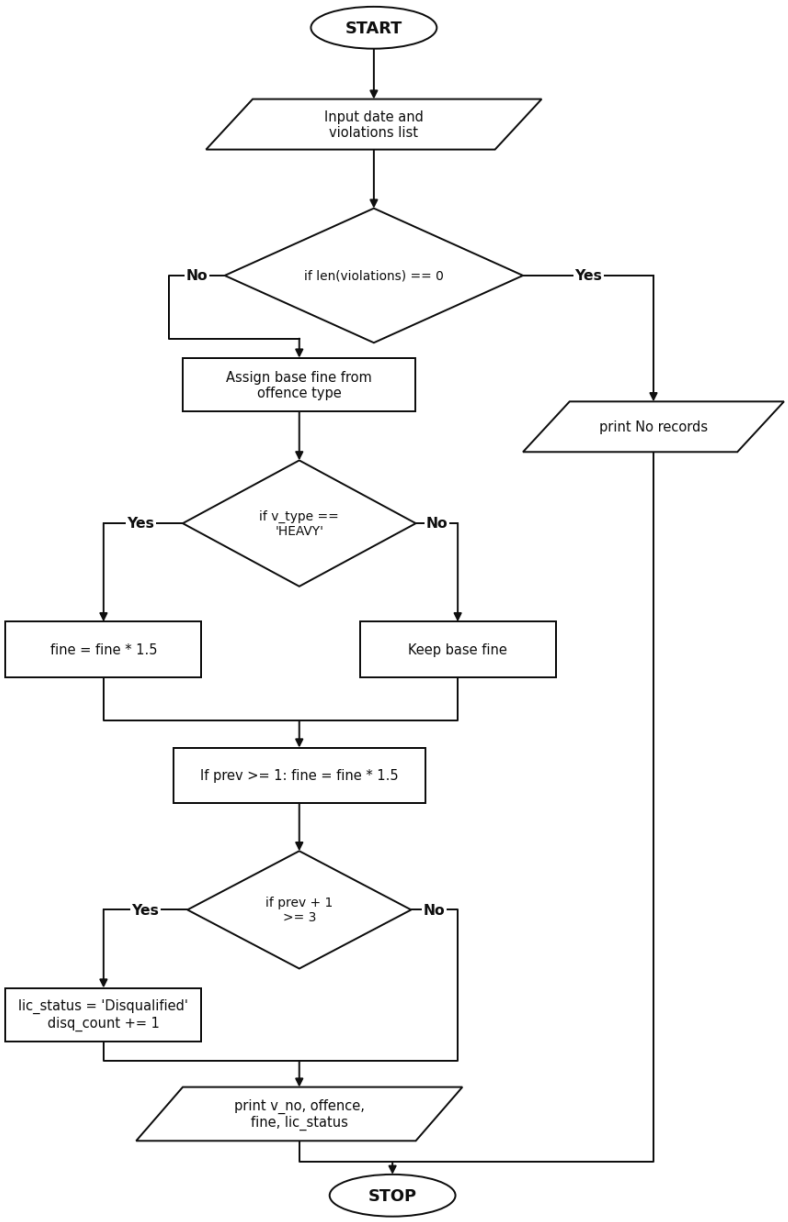
Assumptions

- Base fines are Rs. 1000 for Helmet, Rs. 1500 for Signal, and Rs. 2000 for Speeding.
- Heavy vehicles and repeat offenders each incur a 1.5x fine multiplier.
- Drivers with 3 or more total offences are disqualified.

Algorithm

1. Start.
2. Read the enforcement batch date.
3. For each violation record, determine the base fine from the offence type.
4. Multiply fine by 1.5 if vehicle category is HEAVY.
5. Multiply fine by 1.5 if previous offences >= 1.
6. Mark license as Disqualified if total offences >= 3.
7. Display violation fines and total penalty collected.
8. Stop.

Flowchart



Pseudocode

START

INPUT date

FOR each (v_no, v_type, offence, prev) IN violations

  SET base fine from offence

  IF v_type == "HEAVY"

    fine = fine * 1.5

  IF prev >= 1

    fine = fine * 1.5

```
    IF prev + 1 >= 3
        lic_status = "Disqualified"
    ELSE
        lic_status = "Active"
    PRINT v_no, v_type, offence, fine, lic_status
PRINT total_fine, disq_count
STOP
```

Source Code

```
date = input("Enter Date: ")

# (Vehicle No, Vehicle Type, Offence, Previous Offences)
violations = [
    ("AP16AB1234", "BIKE", "HELMET", 0),
    ("AP39CD5678", "CAR", "SPEEDING", 1),
    ("AP16EF9012", "HEAVY", "SIGNAL", 2),
    ("AP07GH3456", "BIKE", "SIGNAL", 0),
    ("AP39JK7890", "CAR", "SPEEDING", 3)
]

total_fine = 0
disq_count = 0

print("--- Traffic Challan Report ---")
for v in violations:
    v_no, v_type, offence, prev = v

    if offence == "HELMET":
        fine = 1000
    elif offence == "SIGNAL":
        fine = 1500
    else:
        fine = 2000

    if v_type == "HEAVY":
        fine = fine * 1.5

    if prev >= 1:
```

```
        fine = fine * 1.5

    if (prev + 1) >= 3:
        lic_status = "Disqualified"
        disq_count = disq_count + 1
    else:
        lic_status = "Active"

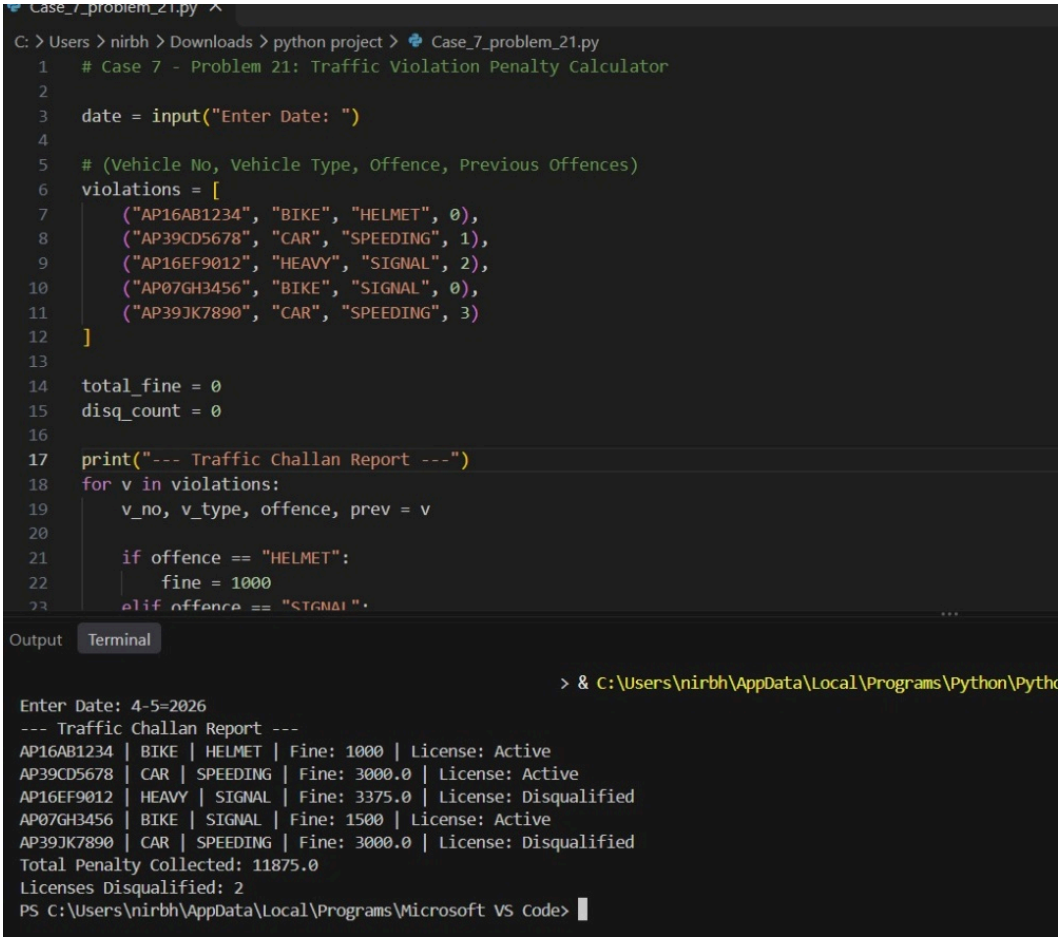
    total_fine = total_fine + fine
    print(v_no, "|", v_type, "|", offence, "| Fine:", fine, "| License:", lic_status)

print("Total Penalty Collected:", total_fine)
print("Licenses Disqualified:", disq_count)
```

Test Cases

1. Enter Date: 20-10-2026
   Output:
   AP16AB1234 | BIKE | HELMET | Fine: 1000 | License: Active
   AP39CD5678 | CAR | SPEEDING | Fine: 3000.0 | License: Active
   AP16EF9012 | HEAVY | SIGNAL | Fine: 3375.0 | License: Disqualified
   AP07GH3456 | BIKE | SIGNAL | Fine: 1500 | License: Active
   AP39JK7890 | CAR | SPEEDING | Fine: 3000.0 | License: Disqualified
   Total Penalty Collected: 11875.0
   Licenses Disqualified: 2

Output



Conclusion

The program calculates traffic challan penalties using conditional rules and identifies repeat offenders for license disqualification.

8.

Tuple-Based Employee Shift Conflict Detection: Represent employee shifts as tuples, compare time intervals, detect overlapping assignments, and identify employees with conflicting schedules.

Sol:

Problem statement

Store employee shift schedules as tuples, compare shift time intervals using nested loops, and detect overlapping shift assignments.

Objectives

- Represent shift records using tuples and unpack shift fields.
- Compare pairs of shifts for the same employee on the same day.
- Detect overlapping time intervals and list affected employees.

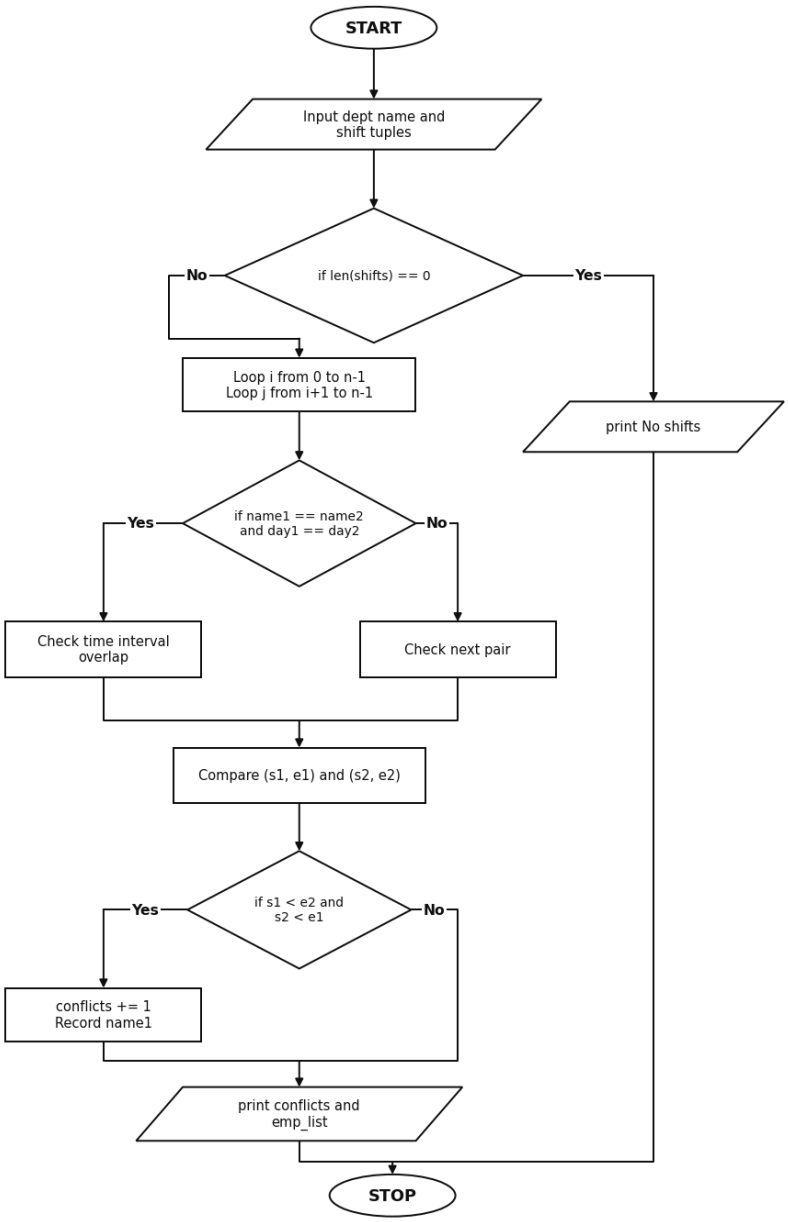
Assumptions

- Shift times are represented in 24-hour format (start hour to end hour).
- Two shifts on the same day overlap if start1 < end2 and start2 < end1.

Algorithm

1. Start.
2. Read the department name and load shift tuples.
3. Use nested loops to compare each shift pair (i, j) where j > i.
4. Check if both shifts belong to the same employee on the same day.
5. Check if start1 < end2 and start2 < end1.
6. If true, print conflict details and record the employee name.
7. Display total conflicts and affected employees.
8. Stop.

Flowchart



Pseudocode

START

INPUT dept

SET conflicts = 0

FOR i = 0 TO len(shifts) - 1

  FOR j = i + 1 TO len(shifts) - 1

    Unpack shifts[i] and shifts[j]

    IF name1 == name2 AND day1 == day2

      IF s1 < e2 AND s2 < e1

```
            conflicts = conflicts + 1
            PRINT name1, day1, id1, id2
PRINT conflicts, emp_list
STOP
```

Source Code

```
dept = input("Enter Department Name: ")

# (Shift ID, Employee Name, Day, Start Hour, End Hour)
shifts = (
   ("SH101", "Arjun", "Monday", 9, 13),
   ("SH102", "Divya", "Monday", 10, 14),
   ("SH103", "Arjun", "Monday", 12, 16),
   ("SH104", "Kiran", "Tuesday", 8, 12),
   ("SH105", "Divya", "Monday", 13, 17),
   ("SH106", "Kiran", "Tuesday", 12, 16)
)

print("--- Checking Shift Conflicts for", dept, "---")
conflicts = 0
emp_list = []

n = len(shifts)
for i in range(n):
   id1, name1, day1, s1, e1 = shifts[i]
   for j in range(i + 1, n):
      id2, name2, day2, s2, e2 = shifts[j]

      if name1 == name2 and day1 == day2:
         if s1 < e2 and s2 < e1:
            conflicts = conflicts + 1
            print("Conflict:", name1, "on", day1, "between", id1, "and", id2)
            if name1 not in emp_list:
               emp_list.append(name1)

print("Total Overlapping Shifts:", conflicts)
print("Employees with Conflicts:", emp_list)
```

Test Cases

1. Enter Department Name: Computer Lab Support

   Output:

   Conflict: Arjun on Monday between SH101 and SH103

   Conflict: Divya on Monday between SH102 and SH105

   Total Overlapping Shifts: 2

   Employees with Conflicts: ['Arjun', 'Divya']

Output

```
# Case 8 - Problem 48: Tuple-Based Employee Shift Conflict Detection

dept = input("Enter Department Name: ")

# (Shift ID, Employee Name, Day, Start Hour, End Hour)
shifts = (
    ("SH101", "Arjun", "Monday", 9, 13),
    ("SH102", "Divya", "Monday", 10, 14),
    ("SH103", "Arjun", "Monday", 12, 16),
    ("SH104", "Kiran", "Tuesday", 8, 12),
    ("SH105", "Divya", "Monday", 13, 17),
    ("SH106", "Kiran", "Tuesday", 12, 16)
)

print("--- Checking Shift Conflicts for", dept, "---")
conflicts = 0
emp_list = []

n = len(shifts)
for i in range(n):
    id1, name1, day1, s1, e1 = shifts[i]
    for j in range(i + 1, n):
```

```
utput  Terminal

PS C:\Users\nirbh\AppData\Local\Programs\Microsoft VS Code> & C:\Users\nirbh\AppData\Local\Programs\Py
Enter Department Name: office
--- Checking Shift Conflicts for office ---
Conflict: Arjun on Monday between SH101 and SH103
Conflict: Divya on Monday between SH102 and SH105
Total Overlapping Shifts: 2
Employees with Conflicts: ['Arjun', 'Divya']
PS C:\Users\nirbh\AppData\Local\Programs\Microsoft VS Code>
```

Conclusion

The program uses tuple unpacking and nested loops to compare shift timings and identify employees with overlapping schedules.

9.

Campus Transport Allocation: Determine the number of buses needed for several routes, account for bus capacities and passenger demand, and identify unserved passengers.

Sol:

Problem statement

Calculate the number of buses needed for campus routes based on student demand and bus seating capacity, allocate available buses, and find unserved passengers.

Objectives

- Read bus seating capacity and route demand records.
- Calculate buses required per route using integer division and remainder.
- Allocate available buses and compute unserved students.

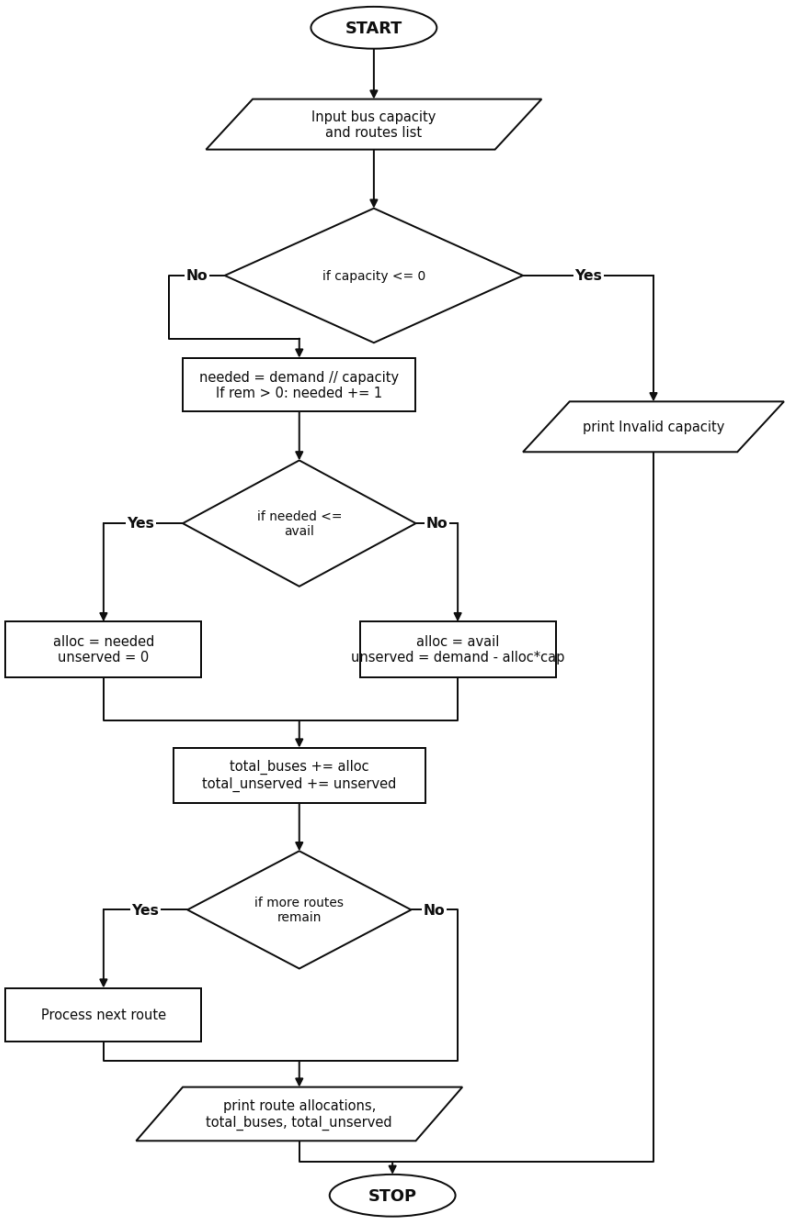
Assumptions

- All buses have identical seating capacity.
- Allocated buses for a route cannot exceed available buses.
- Unserved students equal passenger demand minus seats allocated.

Algorithm

1. Start.
2. Read seating capacity per bus.
3. For each route, read route name, passenger demand, and available buses.
4. Calculate buses needed using demand // capacity (plus 1 if remainder > 0).
5. Allocate buses up to available count.
6. Compute unserved students if buses needed exceed available buses.
7. Display route allocation summary and stop.

Flowchart



Pseudocode

START

INPUT capacity

FOR each [r_name, demand, avail] IN routes

    needed = demand // capacity

    IF demand % capacity > 0

        needed = needed + 1

    IF needed <= avail

        alloc = needed

```
        unserved = 0
    ELSE
        alloc = avail
        unserved = demand - (alloc * capacity)
    PRINT r_name, demand, needed, alloc, unserved
PRINT total_buses, total_unserved
STOP
```

Source Code

```
capacity = int(input("Enter Bus Seating Capacity: "))

# [Route Name, Student Demand, Available Buses]
routes = [
    ["Route-1 Benz Circle", 185, 4],
    ["Route-2 Gannavaram", 140, 2],
    ["Route-3 Eluru Road", 210, 4],
    ["Route-4 Ibrahimpatnam", 95, 2],
    ["Route-5 Gudivada", 160, 3]
]

total_buses = 0
total_unserved = 0

print("--- Campus Bus Allocation ---")
for r in routes:
    r_name = r[0]
    demand = r[1]
    avail = r[2]

    needed = demand // capacity
    if (demand % capacity) > 0:
        needed = needed + 1

    if needed <= avail:
        alloc = needed
        unserved = 0
    else:
        alloc = avail
```

```
        unserved = demand - (alloc * capacity)

    total_buses = total_buses + alloc
    total_unserved = total_unserved + unserved
    print(r_name, "-> Demand:", demand, "Needed:", needed, "Allocated:", alloc, "Unserved:",
unserved)

print("Total Buses Allocated:", total_buses)
print("Total Unserved Students:", total_unserved)
```

Test Cases

1. Enter Bus Seating Capacity: 50

   Output:

   Route-1 Benz Circle -> Demand: 185 Needed: 4 Allocated: 4 Unserved: 0

   Route-2 Gannavaram -> Demand: 140 Needed: 3 Allocated: 2 Unserved: 40

   Route-3 Eluru Road -> Demand: 210 Needed: 5 Allocated: 4 Unserved: 10

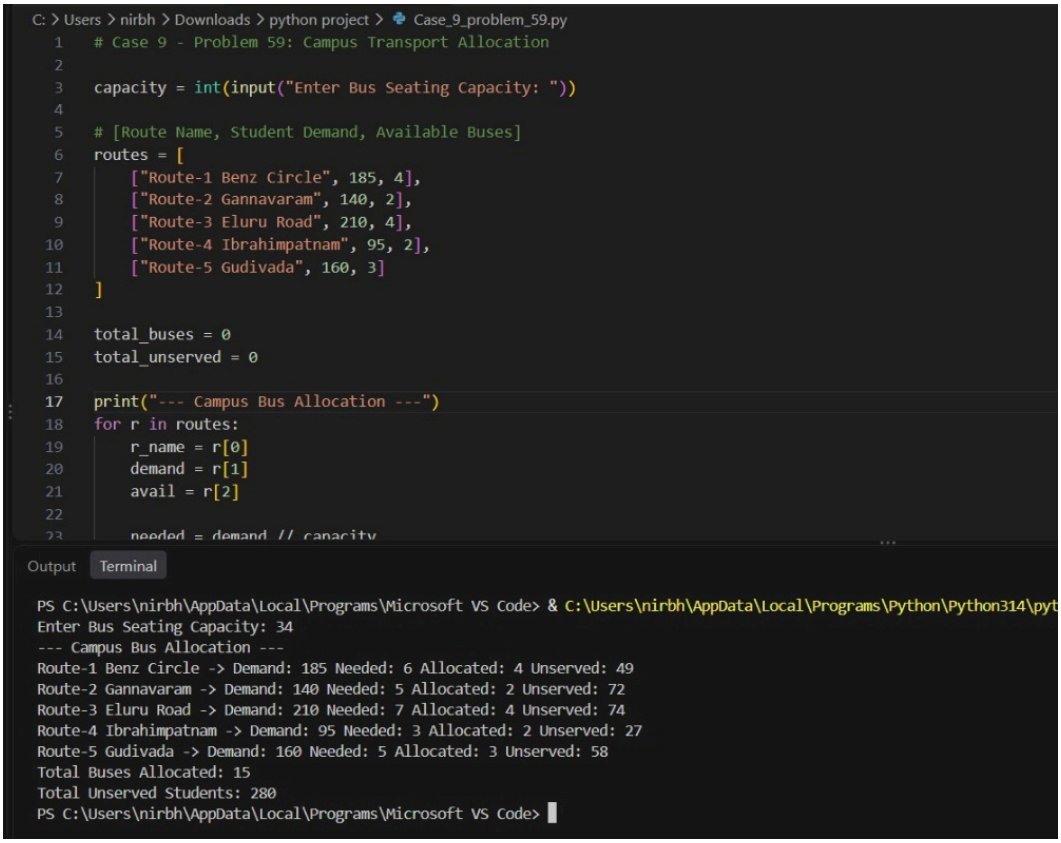
   Route-4 Ibrahimpatnam -> Demand: 95 Needed: 2 Allocated: 2 Unserved: 0

   Route-5 Gudivada -> Demand: 160 Needed: 4 Allocated: 3 Unserved: 10

   Total Buses Allocated: 15

   Total Unserved Students: 60

Output



Conclusion

The program calculates route-wise bus requirements, allocates available buses, and identifies unserved passengers.

10.

Retail Order Dispatch and Return Analysis: Process order quantities, dispatch records, and returned quantities; calculate outstanding orders, update inventory, and detect inconsistent records.

Sol:

Problem statement

Validate retail order dispatch and return records, calculate pending order quantities and updated stock levels, and flag inconsistent records.

Objectives

- Process order, dispatch, and return quantities from nested lists.
- Detect invalid records where dispatched exceeds ordered/stock or returned exceeds dispatched.

- Compute pending units and updated stock for valid orders.

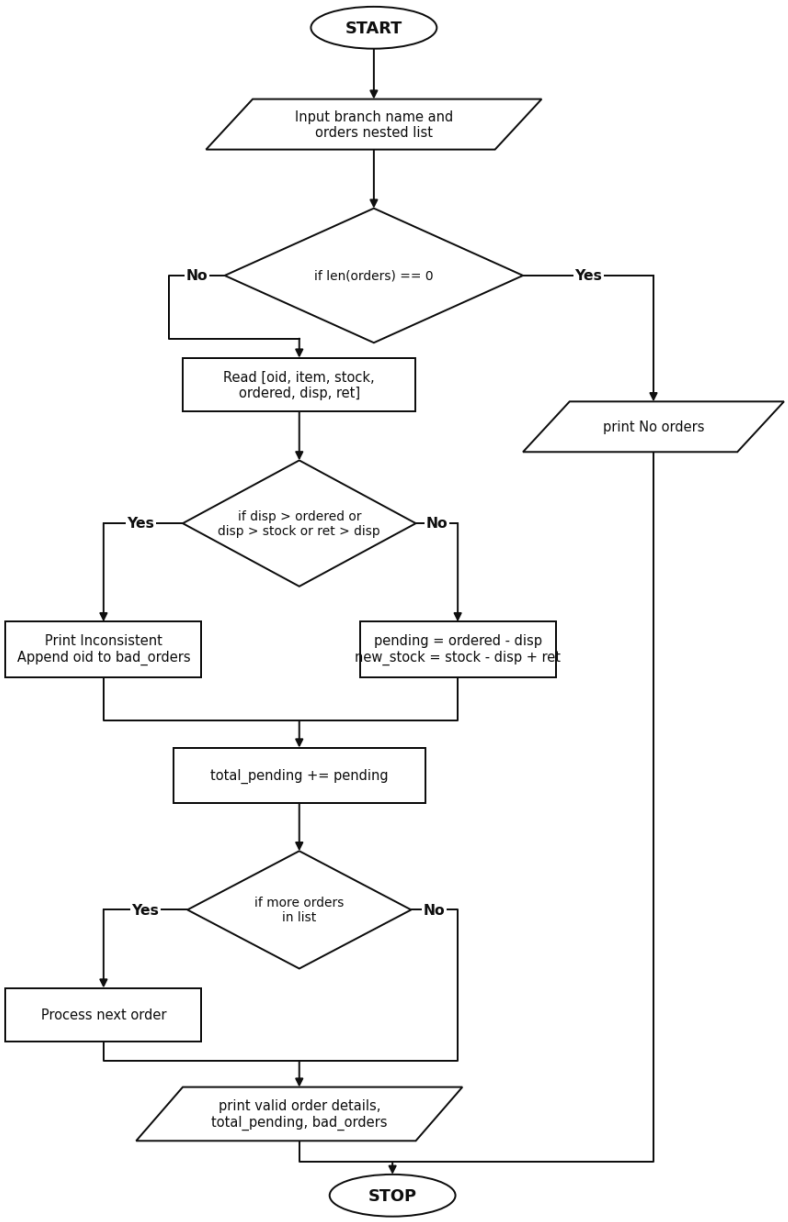
Assumptions

- Dispatched units cannot exceed ordered units or opening stock.
- Returned units cannot exceed dispatched units.
- Updated stock equals opening stock minus dispatched plus returned.

Algorithm

1. Start.
2. Read the retail branch name and load order records.
3. For each order, check if dispatched > ordered, dispatched > stock, or returned > dispatched.
4. If any condition is violated, mark the order as inconsistent.
5. Otherwise compute pending = ordered - dispatched and new_stock = stock - dispatched + returned.
6. Display order status, total pending units, and inconsistent order IDs.
7. Stop.

Flowchart



Pseudocode

START

INPUT branch

FOR each [oid, item, stock, ordered, disp, ret] IN orders

  IF disp > ordered OR disp > stock OR ret > disp

    PRINT oid, "Inconsistent Record"

    Append oid to bad_orders

  ELSE

    pending = ordered - disp

```
        new_stock = stock - disp + ret
        PRINT oid, item, pending, new_stock
PRINT total_pending, bad_orders
STOP
```

Source Code

```
branch = input("Enter Branch Name: ")

# [Order ID, Item, Stock, Ordered, Dispatched, Returned]
orders = [
    ["ORD501", "Laptop Bag", 100, 40, 35, 5],
    ["ORD502", "USB Drive", 80, 50, 50, 2],
    ["ORD503", "Webcam", 60, 30, 35, 0],
    ["ORD504", "HDMI Cable", 120, 45, 40, 8],
    ["ORD505", "Power Bank", 50, 25, 20, 25]
]

bad_orders = []
total_pending = 0

print("--- Order Dispatch & Return Audit ---")
for rec in orders:
    oid = rec[0]
    item = rec[1]
    stock = rec[2]
    ordered = rec[3]
    disp = rec[4]
    ret = rec[5]

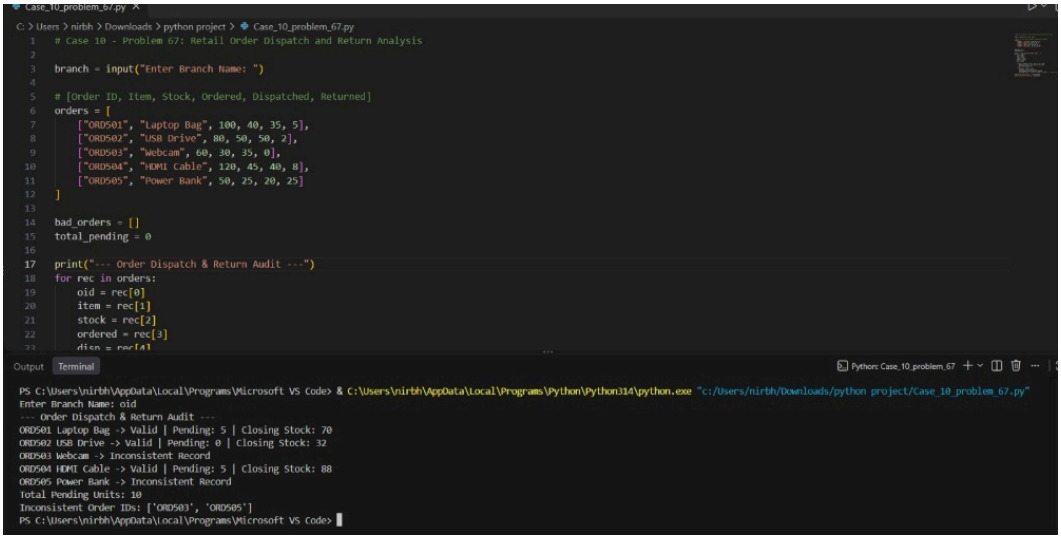
    if disp > ordered or disp > stock or ret > disp:
        print(oid, item, "-> Inconsistent Record")
        bad_orders.append(oid)
    else:
        pending = ordered - disp
        new_stock = stock - disp + ret
        total_pending = total_pending + pending
        print(oid, item, "-> Valid | Pending:", pending, "| Closing Stock:", new_stock)
```

print("Total Pending Units:", total_pending)
print("Inconsistent Order IDs:", bad_orders)

Test Cases

1. Enter Branch Name: Vijayawada Central Store
   Output:
   ORD501 Laptop Bag -> Valid | Pending: 5 | Closing Stock: 70
   ORD502 USB Drive -> Valid | Pending: 0 | Closing Stock: 32
   ORD503 Webcam -> Inconsistent Record
   ORD504 HDMI Cable -> Valid | Pending: 5 | Closing Stock: 88
   ORD505 Power Bank -> Inconsistent Record
   Total Pending Units: 10
   Inconsistent Order IDs: ['ORD503', 'ORD505']

Output



Conclusion

The program validates order dispatch and return records, updates stock balances, and flags inconsistent orders.